

1 Online Resource 1: JAX Implementation Details
2 Supporting Information for “When Inverse Physics-Informed Neural
3 Networks Become Practically Identifiable: A Case Study in Synaptic
4 Dopamine Transport”

5 Emmanuel Hackman

6 Huiqing Zhu

School of Mathematics and Natural Sciences,
University of Southern Mississippi, Hattiesburg, MS, USA

Corresponding author: `Emmanuel.Hackman@usm.edu`

7 We summarize the key JAX code blocks used to construct and train the PINN models
8 reported in the main text. The full reproducible implementation, including plotting helpers,
9 the analytical-solution validation, and the explicit two-dimensional finite-difference reference
10 solver, is publicly available in the GitHub repository cited in the Code Availability statement
11 of the main text (<https://github.com/Hachero98/dopamine-PINN->). The implementation
12 uses Flax NNX for the network module, Optax for the first-order Adam phase, and jaxopt for
13 the second-order L-BFGS phase. All collocation-point arrays are JAX-native (`jnp.ndarray`)
14 and the training step is JIT-compiled via `nnx.jit`.

S1.1 Network, residual, and collocation sampling

The MLP takes $(x, y, t) \in \mathbb{R}^3$ and returns a scalar concentration. Spatial and temporal derivatives needed for the PDE residual are obtained by composing `jax.grad` on the network output; `jax.vmap` batches the per-point residual across all N_r collocation points without an explicit Python loop.

```
import jax, jax.numpy as jnp
import numpy as np
from flax import nnx
import optax
from jaxopt import LBFGS

jax.config.update("jax_enable_x64", True)

D_TRUE, K_TRUE = 0.32, 0.05    # diffusion (mu m^2/ms), reuptake (1/ms)
L, T           = 5.0, 20.0     # side length (mu m), simulation (ms)
SIGMA, CO      = 0.5, 1.0     # release width (mu m), peak (mu M)

class MLP(nnx.Module):
    def __init__(self, layers, *, rngs):
        # Each Linear layer is its own named attribute (lin_0, lin_1, ...)
        # so Flax NNX treats them as named submodules without nnx.List.
        self.n_layers = len(layers) - 1
        for i in range(self.n_layers):
            setattr(self, f"lin_{i}",
                    nnx.Linear(layers[i], layers[i + 1],
                               kernel_init=nnx.initializers.glorot_normal(),
```

```

41         rngs=rngs))
42     def __call__(self, xyt):
43         h = xyt
44         for i in range(self.n_layers - 1):
45             h = jnp.tanh(getattr(self, f"lin_{i}")(h))
46         return getattr(self, f"lin_{self.n_layers - 1}")(h)
47
48     def C_at(model, x, y, t):
49         return model(jnp.stack([x, y, t]))[0]
50
51     # PDE residual at one point: dC/dt - D Laplacian(C) + k C
52     def residual_one(model, x, y, t, D, k):
53         dC_dt = jax.grad(C_at, argnums=3)(model, x, y, t)
54         d2C_dx2 = jax.grad(lambda u: jax.grad(C_at, argnums=1)
55                             (model, u, y, t))(x)
56         d2C_dy2 = jax.grad(lambda v: jax.grad(C_at, argnums=2)
57                             (model, x, v, t))(y)
58         return dC_dt - D * (d2C_dx2 + d2C_dy2) + k * C_at(model, x, y, t)

```

59 S1.2 Forward problem

60 The forward PINN treats D and k as known constants and learns the concentration field
61 $\hat{C}(x, y, t; \theta)$ that satisfies the PDE residual together with the IC and Neumann BC losses.
62 The Adam phase runs in Optax; the L-BFGS phase runs through jaxopt on the same NNX
63 parameter pytree, exposed via `nnx.split` and reassembled via `nnx.merge`.

```

64 rngs = nnx.Rngs(1234)

```

```

65 model      = MLP([3, 64, 64, 64, 64, 1], rngs=rngs)
66 optimizer = nnx.Optimizer(model, optax.adam(1e-3), wrt=nnx.Param)
67
68 def forward_loss(model, pts):
69     res = jax.vmap(residual_one,
70                    in_axes=(None, 0, 0, 0, None, None))(
71         model, pts["x_r"], pts["y_r"], pts["t_r"], D_TRUE, K_TRUE)
72     L_r = jnp.mean(res ** 2)
73     # IC and BC losses computed analogously (omitted for brevity)
74     return 10.0 * L_r + L_ic + L_bc
75
76 @nnx.jit
77 def adam_step(model, optimizer, pts):
78     loss, grads = nnx.value_and_grad(forward_loss)(model, pts)
79     optimizer.update(model, grads)
80     return loss
81
82 for it in range(20_000):
83     adam_step(model, optimizer, pts)
84
85 # L-BFGS phase: NNX state becomes the parameter pytree for jaxopt
86 gdef, state = nnx.split(model)
87 def lbfgs_loss(params):
88     return forward_loss(nnx.merge(gdef, params), pts)
89 result = LBFGS(fun=lbfgs_loss, maxiter=5_000, tol=1e-9).run(state)
90 model  = nnx.merge(gdef, result.params)

```

S1.3 Inverse problem

In JAX, the diffusion coefficient and the reuptake rate are exposed as `nnx.Param` attributes of an `InverseMLP` module, so they live in the same parameter pytree as the network weights and are optimized jointly without a dedicated “external trainable variables” interface. Log-parametrization keeps D and k strictly positive.

```
class InverseMLP(nnx.Module):
    def __init__(self, layers, D0, k0, *, rngs):
        self.mlp = MLP(layers, rngs=rngs)
        self.D_log = nnx.Param(jnp.asarray(np.log(D0)))
        self.k_log = nnx.Param(jnp.asarray(np.log(k0)))
    def __call__(self, xyt):
        return self.mlp(xyt)
    @property
    def D(self): return jnp.exp(self.D_log.value)
    @property
    def k(self): return jnp.exp(self.k_log.value)

def inverse_loss(model, pts, obs):
    D, k = model.D, model.k
    res = jax.vmap(residual_one,
                    in_axes=(None, 0, 0, 0, None, None))(
        model.mlp, pts["x_r"], pts["y_r"], pts["t_r"], D, k)
    L_r = jnp.mean(res ** 2)
    C_pred = jax.vmap(C_at, in_axes=(None, 0, 0, 0))(
        model.mlp, obs["x_d"], obs["y_d"], obs["t_d"])
    L_d = jnp.mean((C_pred - obs["C_d"]) ** 2)
```

```

117     # IC and BC losses computed analogously (omitted for brevity)
118     return 10.0 * L_r + L_ic + L_bc + L_d
119
120 # (obs_x, obs_y, obs_t, obs_C) drawn from FD reference at 2% noise.
121 # Deliberately offset initial guesses D_0 = 0.30, k_0 = 0.04.
122 model      = InverseMLP([3, 64, 64, 64, 64, 1],
123                          D0=0.30, k0=0.04, rngs=nnx.Rngs(1234))
124 optimizer = nnx.Optimizer(model, optax.adam(1e-3), wrt=nnx.Param)
125
126 @nnx.jit
127 def step(model, optimizer, pts, obs):
128     loss, grads = nnx.value_and_grad(inverse_loss)(model, pts, obs)
129     optimizer.update(model, grads)
130     return loss
131
132 history = []
133 for it in range(20_000):
134     step(model, optimizer, pts, obs)
135     if it % 500 == 0:
136         history.append((it, float(model.D), float(model.k)))
137
138 # L-BFGS phase identical in shape to the forward problem
139 gdef, state = nnx.split(model)
140 def lbfgs_loss_inv(params):
141     return inverse_loss(nnx.merge(gdef, params), pts, obs)
142 result = LBFGS(fun=lbfgs_loss_inv, maxiter=5_000, tol=1e-9).run(state)
143 model  = nnx.merge(gdef, result.params)

```

144 After training, the physical parameters are recovered by reading `model.D` and `model.k`, which
145 apply `jnp.exp` to the underlying log-parameterized variables. The trajectory of (D, k) logged
146 every 500 Adam iterations is persisted to `variables.dat` for post-hoc visualization (Fig 4
147 of the main text).